Tarea 1.1: Cambiar el tipo de una instancia con AWS CLI

UD01 - Servicios computacionales basados en máquinas virtuales

Administració de recursos de computació en el núvol

Alumno: Pau López Núñez **Iniciales:** PLN **Fecha:** 21/10/2026

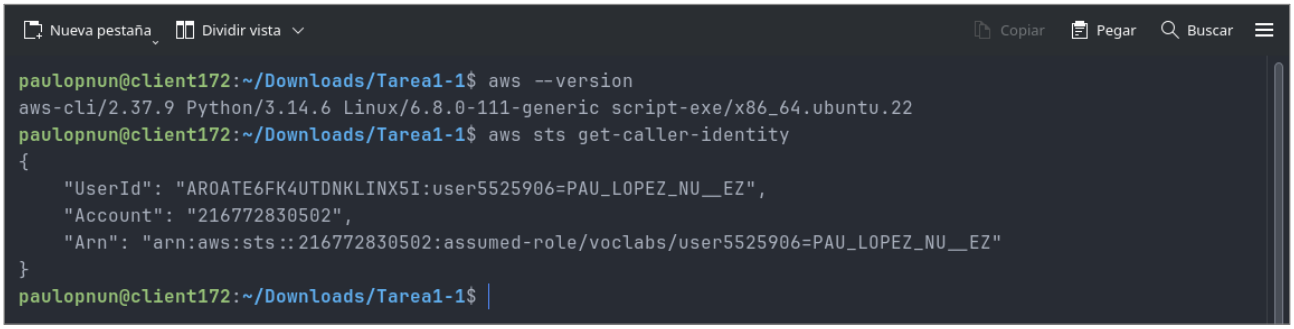
Resumen

Instancia de pruebas **t11-pln**, etiquetas `Propietario=PLN` y `Proyecto=UD01`, región us-east-1, AWS CLI v2.37.9. Las pruebas de las Partes 1, 5 y 7 son capturas de pantalla de la terminal (Konsole) con la instancia `i-061d350c0a7fa7075`; el proceso manual de la Parte 2 se hizo antes con otra t11-pln (`i-0101ee87523d6678a`) y se muestra la salida de la terminal como texto. El script `cambia_tipo_ec2.sh` se incluye completo en el anexo.

Prueba	Qué se ejecuta	Esperado	Obtenido
P1	Sin parámetros	1	1
P2	ID que no existe	2	2
P3	t3.xlarge	5, sin detener	5 (sigue running)
P4	t3.small, respondiendo n	6, sin cambios	6 (sigue running t3.micro)
P5	t3.small, en ejecución	0, running t3.small	0
P6	-y t3.micro, detenida	0, stopped t3.micro	0
E1-E4	Pruebas de ampliación	3, 5, 0, 1	3, 5, 0, 1

Parte 1: Preparación

Comprobación de la CLI y de las credenciales del Learner Lab:



Instancia de pruebas t11-pln lanzada con `run-instances` (AMI de Ubuntu 26.04 resuelta desde SSM), con su nombre, ID, estado, tipo y etiqueta Propietario:



Parte 2: El proceso, paso a paso

Datos iniciales, intento de cambio con la instancia en marcha (error `IncorrectInstanceState`) y el proceso completo a t3.small y de vuelta a t3.micro, midiendo con `time` las esperas. La consulta muestra estado, tipo, arquitectura, zona, IP pública y, añadida, la IP privada.

```
$ ID=i-0101ee87523d6678a
$ aws ec2 describe-instances --instance-ids $ID --query "$Q" --output text
running t3.micro    x86_64  us-east-1a  54.166.48.196   172.31.95.12
$ aws ec2 modify-instance-attribute --instance-id $ID --instance-type Value=t3.small
aws: [ERROR]: An error occurred (IncorrectInstanceState) when calling the ModifyInstanceAttribute
operation: The instance 'i-0101ee87523d6678a' is not in the 'stopped' state.

$ aws ec2 stop-instances --instance-ids $ID
{
    "StoppingInstances": [
        {
            "InstanceId": "i-0101ee87523d6678a",
            "CurrentState": {
                "Code": 64,
                "Name": "stopping"
            },
            "PreviousState": {
                "Code": 16,
                "Name": "running"
            }
        }
    ]
}
$ time aws ec2 wait instance-stopped --instance-ids $ID
real    0m17,090s
user    0m1,258s
sys 0m0,127s
$ aws ec2 modify-instance-attribute --instance-id $ID --instance-type Value=t3.small
$ aws ec2 start-instances --instance-ids $ID
{
    "StartingInstances": [
        {
            "InstanceId": "i-0101ee87523d6678a",
            "CurrentState": {
```

```
                "Code": 0,
                "Name": "pending"
            },
            "PreviousState": {
                "Code": 80,
                "Name": "stopped"
            }
        }
    ]
}
$ time aws ec2 wait instance-running --instance-ids $ID
real    0m16,754s
user    0m1,002s
sys 0m0,110s
$ aws ec2 describe-instances --instance-ids $ID --query "$Q" --output text
running t3.small    x86_64  us-east-1a  13.221.82.49    172.31.95.12

$ aws ec2 stop-instances --instance-ids $ID
{
    "StoppingInstances": [
        {
            "InstanceId": "i-0101ee87523d6678a",
            "CurrentState": {
                "Code": 64,
                "Name": "stopping"
            },
            "PreviousState": {
                "Code": 16,
                "Name": "running"
            }
        }
    ]
}
$ time aws ec2 wait instance-stopped --instance-ids $ID
real    0m16,674s
user    0m0,916s
sys 0m0,108s
$ aws ec2 modify-instance-attribute --instance-id $ID --instance-type Value=t3.micro
$ aws ec2 start-instances --instance-ids $ID
{
    "StartingInstances": [
        {
            "InstanceId": "i-0101ee87523d6678a",
            "CurrentState": {
                "Code": 0,
                "Name": "pending"
            },
            "PreviousState": {
                "Code": 80,
                "Name": "stopped"
            }
        }
    ]
}
$ time aws ec2 wait instance-running --instance-ids $ID
real    0m16,698s
user    0m0,928s
sys 0m0,127s
$ aws ec2 describe-instances --instance-ids $ID --query "$Q" --output text
running t3.micro    x86_64  us-east-1a  13.221.162.203  172.31.95.12
```

Fase	De t3.micro a t3.small	De t3.small a t3.micro
wait instance-stopped	17,1 s	16,7 s
wait instance-running	16,8 s	16,7 s
IP pública resultante	Pasa de 54.166.48.196 a 13.221.82.49	Pasa de 13.221.82.49 a 13.221.162.203
IP privada	172.31.95.12 (igual)	172.31.95.12 (igual)

Para pensar

¿Ha cambiado el ID de la instancia? No. Sigue siendo `i-0101ee87523d6678a`: es la misma instancia (mismo volumen EBS raíz, mismas etiquetas); solo cambia el hardware virtual en el que se ejecuta.

¿Y su IP pública? Sí, cambia cada vez que se detiene y arranca: 54.166.48.196, luego 13.221.82.49 y luego 13.221.162.203. La IP pública automática no pertenece a la instancia, AWS la libera al detenerla y le asigna otra del conjunto de Amazon al arrancar.

¿Y la privada? No: 172.31.95.12 en todos los casos. La IP privada está asociada a la interfaz de red (ENI), que se conserva mientras la instancia exista.

¿Qué le pasaría a un servicio cuyos clientes se conectan a esa IP pública? Dejaría de funcionar: los clientes seguirían apuntando a la IP antigua, que ya no es nuestra (incluso podría acabar en otra cuenta). Además estaría caído durante el cambio (unos 35 s de esperas más el arranque del sistema operativo). Para evitarlo se usa una **IP elástica**, un nombre DNS (Route 53) que se actualice, o un balanceador de carga delante de varias instancias, que además permite hacer el cambio sin cortar el servicio.

¿Por qué EC2 no permite cambiar el tipo con la instancia en marcha? El tipo determina el hardware real que se le asigna: número de vCPU, memoria, rendimiento de red y EBS, e incluso la familia de procesador (t3 Intel, t3a AMD). Normalmente el nuevo tipo no cabe en el mismo servidor físico, así que AWS tiene que colocar la instancia en otro host, y eso solo puede hacerse con la máquina parada (el volumen EBS se desconecta y se vuelve a conectar en el nuevo host). Además, el sistema operativo detecta CPU y memoria al arrancar; no está preparado para que le cambien el hardware en caliente.

Partes 3 y 4: El script cambia_tipo_ec2.sh

El script implementa el nivel completo (Parte 4). Su funcionamiento, en orden:

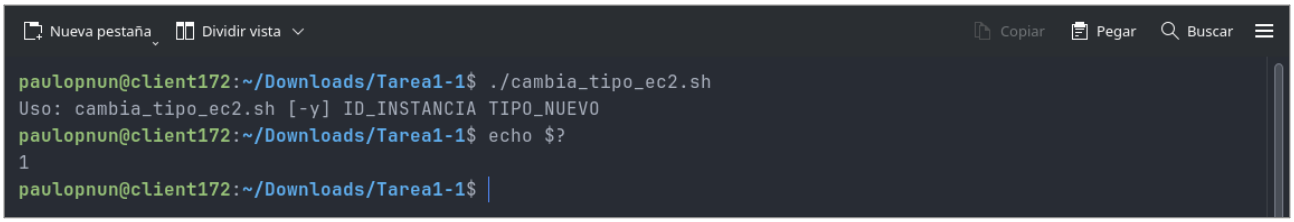
Paso	Qué hace	Código
Opciones	getopts ':y'; una opción desconocida o un número de parámetros distinto de 2 muestran el uso	1
Existe	Una sola llamada a describe-instances obtiene estado, tipo, etiqueta Propietario y Name	2
Es mía	La etiqueta Propietario debe ser igual a PROPIETARIO="PLN"	3
Estado	Rechaza terminated y shutting-down	4
Mismo tipo	Si el tipo pedido es el actual, informa y termina	0
Tipo válido	El tamaño (${TIPO#*.}) debe ser nano, micro, small, medium o large, y describe-instance-types debe aceptar el tipo. Todo antes de detener nada	5
Confirmación	Sin -y muestra ID, nombre, estado, tipo actual y nuevo, avisa de la parada y pide 's'	6
Cambio	Si estaba running: stop + wait stopped; modify-instance-attribute; si estaba running: start + wait running. Si estaba detenida, la deja detenida	7 si falla AWS
Verificación	Vuelve a consultar AWS el tipo y el estado finales, comprueba que el tipo coincide y muestra el tiempo total con SECONDS	0

Decisiones a destacar: el tamaño se valida **antes** que la existencia del tipo y antes de cualquier parada, para que un tipo mayor que large nunca llegue a arrancar (el Learner Lab terminaría la instancia). Si la instancia está en estado `stopping`, el script también espera a que se detenga antes de modificarla, para evitar el error IncorrectInstanceState. El código fuente completo está en el anexo.

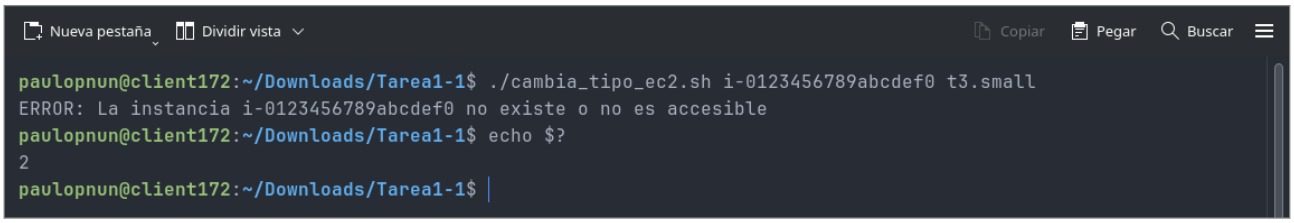
Parte 5: Pruebas

Cada prueba muestra el comando, su salida y el código de salida con echo $?. En P4 y P5 se ve la respuesta escrita en la confirmación (n / s). Tras P3 y P4 se muestra el estado de la instancia para demostrar que no ha cambiado.

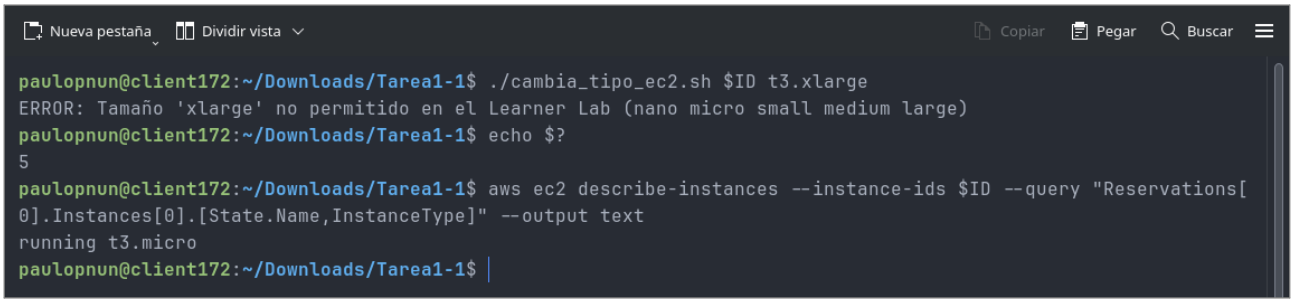
P1 Sin parámetros



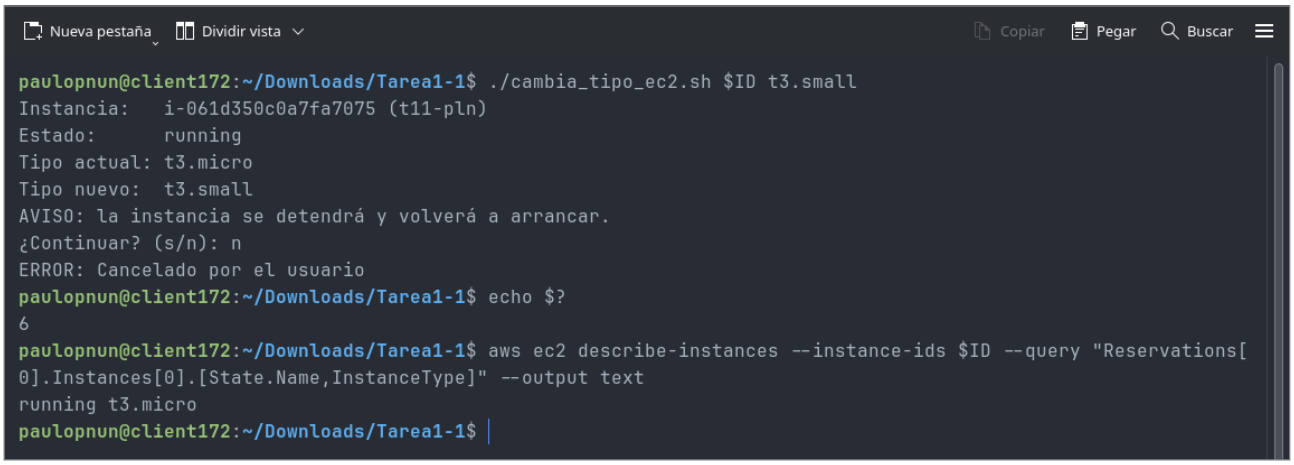
P2 ID que no existe



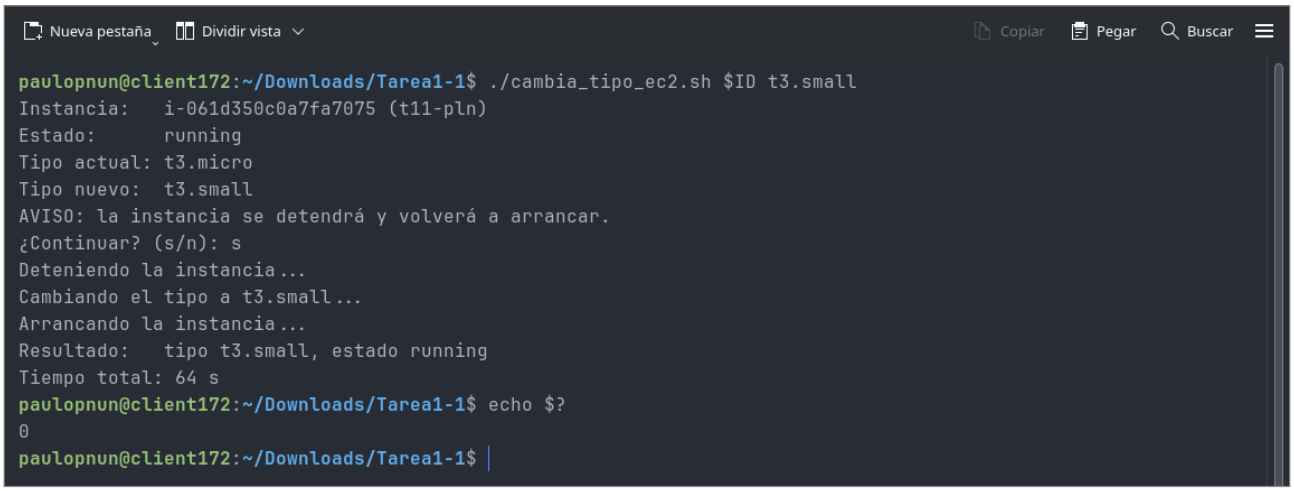
P3 t3.xlarge (tamaño no permitido): código 5 y la instancia sigue igual



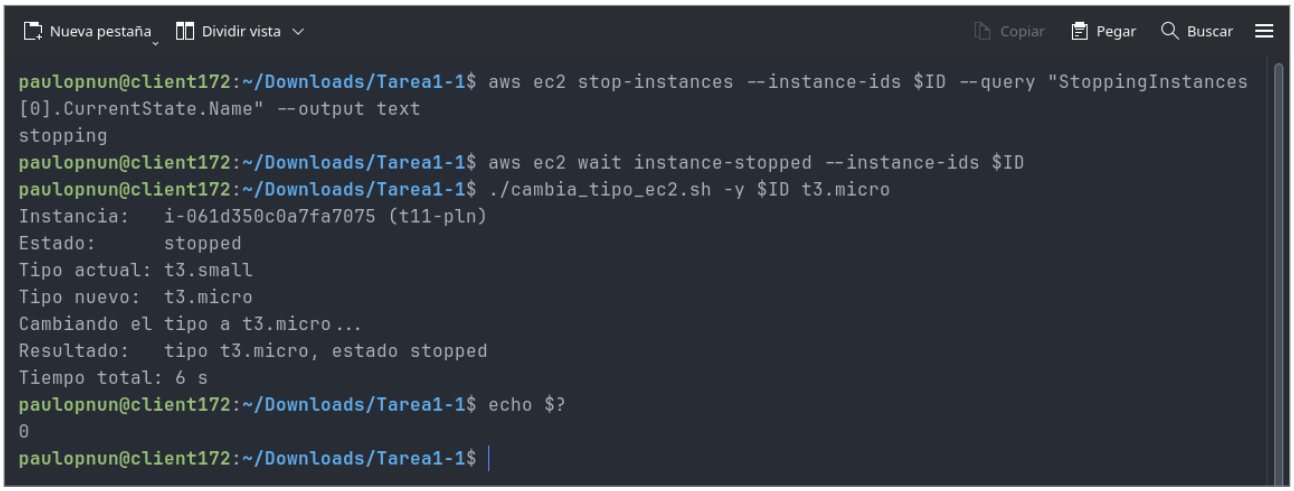
P4 t3.small respondiendo n: código 6 y la instancia sigue igual



P5 t3.small con la instancia en ejecución



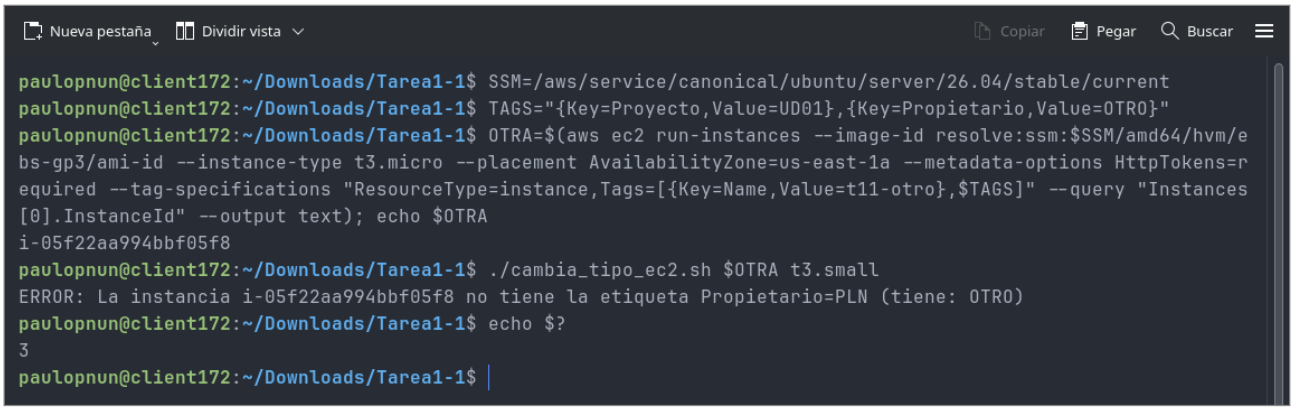
P6 t3.micro con -y y la instancia detenida



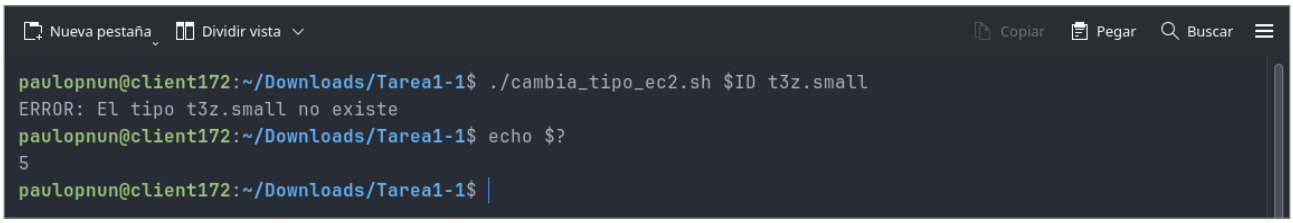
Pruebas de ampliación

Pruebas adicionales para casos que no aparecen en P1-P6: una instancia que no es mía (código 3), un tipo con formato válido que no existe, el código 0 de nada que hacer cuando el tipo ya es el pedido y una opción desconocida. Para E1 se lanza una instancia temporal t11-otro con la etiqueta Propietario=OTRO, que se termina después de la prueba.

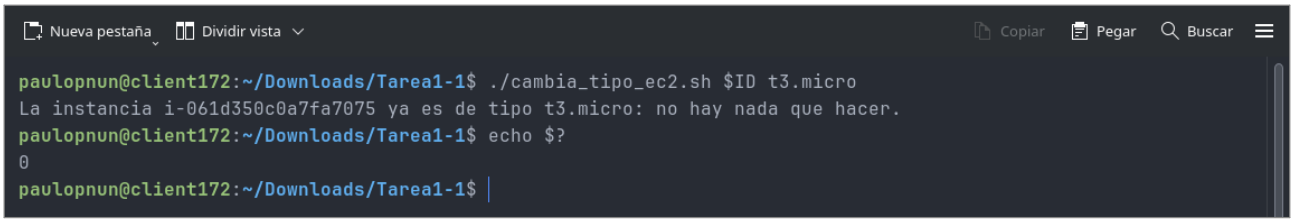
E1 Instancia de otro propietario



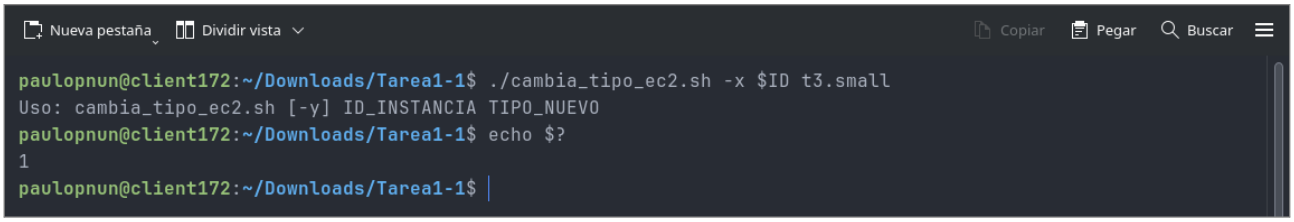
E2 Tipo inexistente



E3 Mismo tipo que el actual

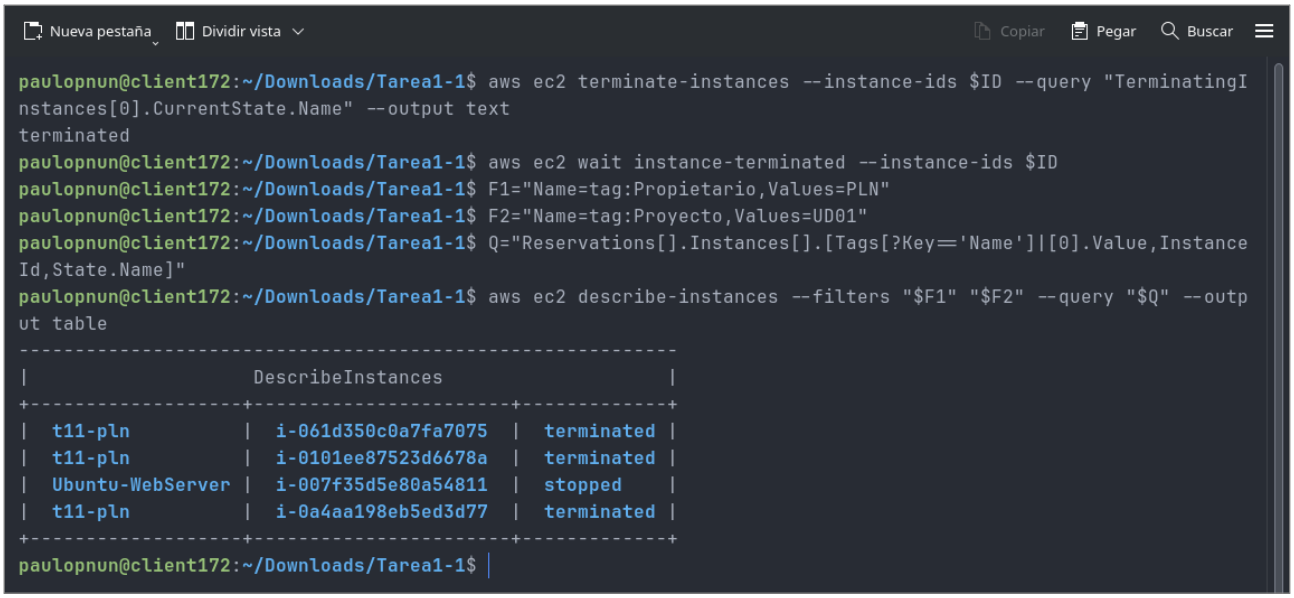


E4 Opción desconocida



Parte 7: Limpieza

Terminación de t11-pln y comprobación con el filtro por etiquetas. Aparecen como terminated las tres t11-pln usadas durante la práctica. El resto de instancias no se ha borrado (se usarán en próximas actividades); se han dejado detenidas para no consumir crédito.



Ampliación por cuenta propia: comprobar la arquitectura

Revisando el script encontré un caso que las comprobaciones de la Parte 4 no cubren: un tipo que existe y cuyo tamaño está permitido, pero que es de otra arquitectura. Por ejemplo, `t4g.small` es de la familia Graviton (procesador ARM, arquitectura `arm64`), mientras que t11-pln usa una AMI de Ubuntu para x86_64. El script original lo daba por válido, detenía la instancia y el cambio fallaba después, dejando el servidor parado. Eso va contra la idea de la Parte 4: fallar antes de tocar nada.

La solución añade una comprobación más antes de la confirmación:

1. La llamada única a `describe-instances` pide también `Architecture` de la instancia (sigue siendo una sola llamada).

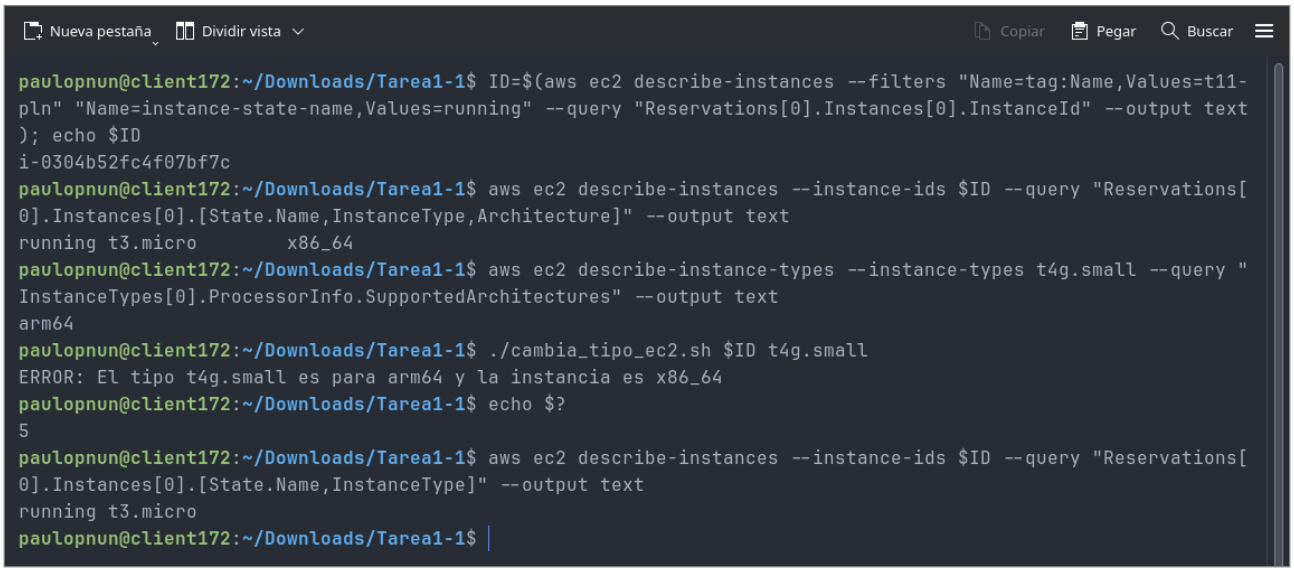
2. La llamada a `describe-instance-types` que ya comprobaba si el tipo existe ahora devuelve además las arquitecturas que admite (`ProcessorInfo.SupportedArchitectures`).

3. Si la arquitectura de la instancia no está entre ellas, el script termina con el código 5 (tipo no válido) sin detener la instancia.

```
ARQ_TIPO=$(aws ec2 describe-instance-types --instance-types "$TIPO_NUEVO" --output text \
  --query "InstanceTypes[0].ProcessorInfo.SupportedArchitectures" 2>/dev/null) \
  || error "El tipo $TIPO_NUEVO no existe" 5
grep -qw -- "$ARQUITECTURA" <<< "$ARQ_TIPO" \
  || error "El tipo $TIPO_NUEVO es para $ARQ_TIPO y la instancia es $ARQUITECTURA" 5
```

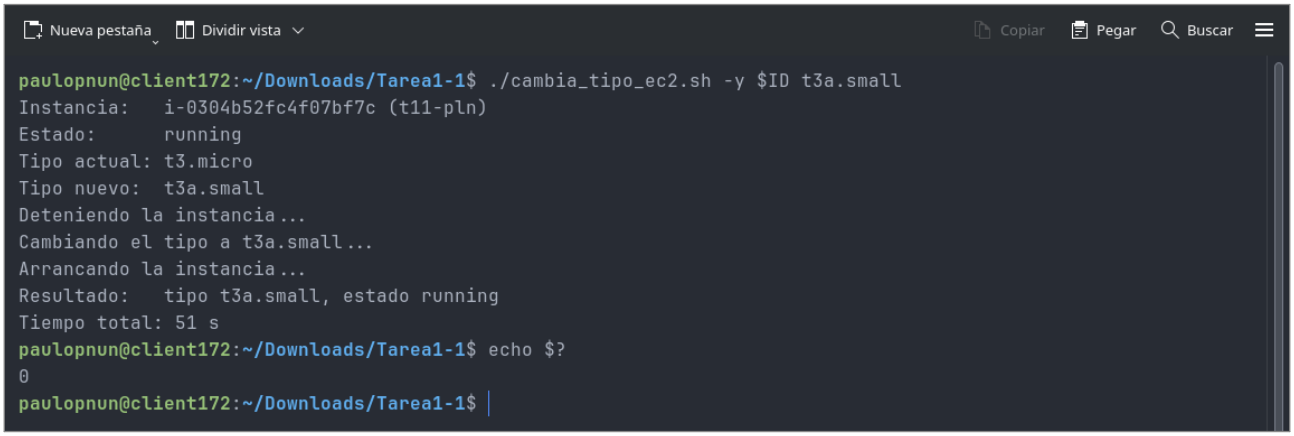
X1 Tipo de otra arquitectura (t4g.small)

La instancia es x86_64 y t4g.small solo admite arm64. El script devuelve el código 5 y, al consultar después, la instancia sigue en ejecución con su tipo original.



X2 Tipo de otra familia con la misma arquitectura (t3a.small)

Prueba de control: t3a es de procesador AMD pero también x86_64, así que la comprobación lo deja pasar y el cambio se hace con normalidad (código 0).



Tras estas pruebas la instancia se terminó, igual que en la Parte 7.

Anexo: cambia_tipo_ec2.sh

Versión final del script, con la ampliación incluida.

```
#!/usr/bin/env bash
# cambia_tipo_ec2.sh - Cambia el tipo de una instancia EC2
# Autor/a: Pau López Núñez (PLN)
# Fecha:    21/10/2026
# Uso:      cambia_tipo_ec2.sh [-y] ID_INSTANCIA TIPO_NUEVO
# Ampliación: comprueba que el tipo nuevo admite la arquitectura de la instancia
set -uo pipefail
export AWS_PAGER=""               # sin paginador: el script no se detiene
# Solo se modifican las instancias con esta etiqueta Propietario
PROPIETARIO="PLN"
I="Reservations[0].Instances[0]"
# Tamaños que el Learner Lab permite arrancar
TAMANOS_PERMITIDOS="nano micro small medium large"
error() {                                 # error MENSAJE CODIGO
  echo "ERROR: $1" >&2
  exit "$2"
}
uso() {
  echo "Uso: $(basename "$0") [-y] ID_INSTANCIA TIPO_NUEVO" >&2
  exit 1
}
# --- 1. Opciones y parámetros ------------------------------------------
CONFIRMAR=1
while getopts ":y" OPCION; do
  case $OPCION in
    y) CONFIRMAR=0 ;;
    *) uso ;;
  esac
done
shift $((OPTIND - 1))
[ $# -eq 2 ] || uso
ID=$1
TIPO_NUEVO=$2
# --- 2. La instancia existe (una sola llamada a describe-instances) ------
DATOS=$(aws ec2 describe-instances --instance-ids "$ID" --output text \
  --query "$I.[State.Name,InstanceType,Architecture,Tags[?Key=='Propietario'].Value | [0],Tags[
?Key=='Name'].Value | [0]]" \
  2>/dev/null) \
  || error "La instancia $ID no existe o no es accesible" 2
IFS=$'\t' read -r ESTADO TIPO_ACTUAL ARQUITECTURA PROP NOMBRE <<< "$DATOS"
# --- 3. La instancia es mía --------------------------------------------
[ "$PROP" = "$PROPIETARIO" ] \
  || error "La instancia $ID no tiene la etiqueta Propietario=$PROPIETARIO (tiene: $PROP)" 3
# --- 4. No está terminada ni terminándose ------------------------------
case $ESTADO in
  terminated|shutting-down)
    error "La instancia $ID está en estado $ESTADO" 4 ;;
esac
# --- 5. El tipo es distinto --------------------------------------------
if [ "$TIPO_NUEVO" = "$TIPO_ACTUAL" ]; then
  echo "La instancia $ID ya es de tipo $TIPO_ACTUAL: no hay nada que hacer."
  exit 0
fi
# --- 6. El tipo existe y su tamaño está permitido ----------------------
TAMANO=${TIPO_NUEVO#*.}
[[ " $TAMANOS_PERMITIDOS " == *" $TAMANO "* ]] \
  || error "Tamaño '$TAMANO' no permitido en el Learner Lab ($TAMANOS_PERMITIDOS)" 5
ARQ_TIPO=$(aws ec2 describe-instance-types --instance-types "$TIPO_NUEVO" --output text \
  --query "InstanceTypes[0].ProcessorInfo.SupportedArchitectures" 2>/dev/null) \
  || error "El tipo $TIPO_NUEVO no existe" 5
# --- Ampliación: el tipo admite la arquitectura de la instancia ----------
# (un t4g es ARM: con una AMI x86_64 la instancia no arrancaría)
grep -qw -- "$ARQUITECTURA" <<< "$ARQ_TIPO" \
  || error "El tipo $TIPO_NUEVO es para $ARQ_TIPO y la instancia es $ARQUITECTURA" 5
# --- 7. Confirmación ---------------------------------------------------
echo "Instancia:   $ID ($NOMBRE)"
echo "Estado:      $ESTADO"
echo "Tipo actual: $TIPO_ACTUAL"
echo "Tipo nuevo:  $TIPO_NUEVO"
if [ "$CONFIRMAR" -eq 1 ]; then
  [ "$ESTADO" = "running" ] && echo "AVISO: la instancia se detendrá y volverá a arrancar."
  read -r -p "¿Continuar? (s/n): " RESPUESTA
```

```
  [ "$RESPUESTA" = "s" ] || error "Cancelado por el usuario" 6
fi
# --- Cambio de tipo ------------------------------------------------------
if [ "$ESTADO" = "running" ]; then
  echo "Deteniendo la instancia..."
  aws ec2 stop-instances --instance-ids "$ID" >/dev/null \
    || error "No se ha podido detener la instancia" 7
fi
# Si estaba deteniéndose o se acaba de detener, esperamos a que pare del todo
if [ "$ESTADO" = "running" ] || [ "$ESTADO" = "stopping" ]; then
  aws ec2 wait instance-stopped --instance-ids "$ID" \
    || error "La instancia no ha llegado a detenerse" 7
fi
echo "Cambiando el tipo a $TIPO_NUEVO..."
aws ec2 modify-instance-attribute --instance-id "$ID" \
  --instance-type "Value=$TIPO_NUEVO" \
  || error "No se ha podido cambiar el tipo" 7
if [ "$ESTADO" = "running" ]; then
  echo "Arrancando la instancia..."
  aws ec2 start-instances --instance-ids "$ID" >/dev/null \
    || error "No se ha podido arrancar la instancia" 7
  aws ec2 wait instance-running --instance-ids "$ID" \
    || error "La instancia no ha llegado a arrancar" 7
fi
# --- Comprobación final en AWS -------------------------------------------
FINAL=$(aws ec2 describe-instances --instance-ids "$ID" --output text \
  --query "$I.[InstanceType,State.Name]") \
  || error "No se ha podido consultar el estado final" 7
read -r TIPO_FINAL ESTADO_FINAL <<< "$FINAL"
echo "Resultado:   tipo $TIPO_FINAL, estado $ESTADO_FINAL"
[ "$TIPO_FINAL" = "$TIPO_NUEVO" ] || error "El tipo final no coincide con el pedido" 7
echo "Tiempo total: ${SECONDS} s"
```